

La Libreria `pathlib` di Python - Guida Pratica

Indice

1. [Introduzione](#)
 2. [Concetti Fondamentali](#)
 3. [Classi Principali](#)
 4. [Operazioni su Percorsi](#)
 5. [Operazioni su File e Cartelle](#)
 6. [Esempi Pratici](#)
 7. [Casi d'Uso Reali](#)
 8. [Esercizi](#)
 9. [pathlib vs os.path](#)
-

Introduzione

`pathlib` è una libreria della **standard library di Python** che offre un modo **moderno e orientato agli oggetti** per lavorare con i percorsi di file e cartelle.

Cosa c'era prima?

```
# Vecchio metodo (ancora funziona ma è antiquato)
import os
percorso = os.path.join('cartella', 'sottocartella', 'file.txt')
if os.path.exists(percorso):
    print(os.path.getsize(percorso))
```

Il modo moderno con `pathlib`:

```
# Nuovo metodo (pulito e orientato agli oggetti)
from pathlib import Path
percorso = Path('cartella') / 'sottocartella' / 'file.txt'
if percorso.exists():
    print(percorso.stat().st_size)
```

Vantaggi di pathlib

- ✓ **Sintassi intuitiva** - Usa `/` per concatenare percorsi
- ✓ **Cross-platform** - Gestisce automaticamente `/` e `\`
- ✓ **Orientato agli oggetti** - Metodi diretti sui percorsi
- ✓ **Più leggibile** - Codice più chiaro e Pythonic
- ✓ **Built-in** - Non serve installare nulla

Concetti Fondamentali

Cos'è un Path?

Un oggetto `Path` rappresenta un **percorso nel sistema di file** (file o cartella). È immutabile e cross-platform.

```
from pathlib import Path

# Creare un Path
p = Path('cartella/file.txt')

# Accesso ai componenti
print(p.name)          # 'file.txt' - nome del file
print(p.stem)          # 'file' - nome senza estensione
print(p.suffix)        # '.txt' - estensione
print(p.parent)        # Path('cartella') - cartella padre
print(p.parts)         # ('cartella', 'file.txt') - tuple dei componenti
```

Percorsi Assoluti vs Relativi

```
from pathlib import Path

# Percorso relativo (relativo alla cartella attuale)
p_rel = Path('cartella/file.txt')
print(p_rel.is_absolute()) # False

# Percorso assoluto (percorso completo dal root)
p_abs = Path('/home/utente/cartella/file.txt')
print(p_abs.is_absolute()) # True

# Convertire a percorso assoluto
p_abs = p_rel.resolve()
```

Percorsi Cross-Platform

```
from pathlib import Path

# pathlib gestisce automaticamente le differenze tra OS
# Unix-like (Linux, Mac)
p_unix = Path('cartella/file.txt') # / è corretto

# Windows
p_win = Path('cartella\\file.txt') # \ sarebbe corretto

# pathlib usa il separatore corretto per l'OS corrente!
print(str(p_unix)) # 'cartella/file.txt' su Unix, 'cartella\file.txt' su Windows
```

Classi Principali

Path - Classe Generale

La classe principale per lavorare con qualsiasi percorso.

```
from pathlib import Path

# Creare Path
p = Path('file.txt')
p = Path('cartella', 'sottocartella', 'file.txt')
p = Path('cartella') / 'sottocartella' / 'file.txt'

# Percorso corrente
p = Path.cwd()

# Home directory
p = Path.home()
```

PureWindowsPath e PurePosixPath

Percorsi "puri" senza accesso al filesystem (utili per manipolazione).

```
from pathlib import PureWindowsPath, PurePosixPath

# Windows
pw = PureWindowsPath('C:\\Users\\file.txt')
print(pw.drive) # 'C:\\'
```

```
print(pw.parent) # PureWindowsPath('C:\\Users')

# POSIX (Linux, Mac)
pp = PurePosixPath('/home/utente/file.txt')
print(pp.parts) # ('/', 'home', 'utente', 'file.txt')
```

WindowsPath e PosixPath

Percorsi specifici dell'OS con accesso al filesystem.

```
from pathlib import WindowsPath, PosixPath

# Automaticamente selezionato in base all'OS
from pathlib import Path # È WindowsPath su Windows, PosixPath su Unix
```

Operazioni su Percorsi

Concatenazione con /

```
from pathlib import Path

base = Path('cartella')
p = base / 'sottocartella' / 'file.txt'
print(p) # Path('cartella/sottocartella/file.txt')

# Funziona anche con stringhe
p = Path('cartella') / 'file.txt'
p = p / 'capitoli' / 'intro.md' # Continua a concatenare
```

Accedere ai Componenti

```
from pathlib import Path

p = Path('/home/utente/Documenti/report.pdf')

print(p.name) # 'report.pdf' - nome completo
print(p.stem) # 'report' - nome senza estensione
print(p.suffix) # '.pdf' - estensione
print(p.suffixes) # ['.tar', '.gz'] - tutte le estensioni (per file.tar.gz)
print(p.parent) # Path('/home/utente/Documenti')
```

```
print(p.parents)      # <PosixPath.parents object> - tutti i parent
print(p.parts)        # ('/', 'home', 'utente', 'Documenti', 'report.pdf')
```

Ottenere Parents

```
from pathlib import Path

p = Path('/home/utente/Documenti/report.pdf')

# Accedere ai parent
print(p.parents[0]) # '/home/utente/Documenti'
print(p.parents[1]) # '/home/utente'
print(p.parents[2]) # '/home'
print(p.parents[3]) # '/'

# Iterare su tutti i parent
for parent in p.parents:
    print(parent)
```

Risolvere Percorsi

```
from pathlib import Path

# Percorso relativo
p_rel = Path('cartella/../altro/file.txt')

# Risolvere (assoluto + normalizzato)
p_abs = p_rel.resolve()
print(p_abs) # Percorso assoluto senza .. e .

# Normalizzare (senza normalizzare a assoluto)
p_norm = p_rel.resolve()
```

Controllare Tipo di Percorso

```
from pathlib import Path

p = Path('file.txt')

print(p.is_absolute()) # False
print(p.is_relative_to(Path('.'))) # True (se dentro cartella corrente)
```

Operazioni su File e Cartelle

Verificare Esistenza

```
from pathlib import Path

p = Path('file.txt')

if p.exists():
    print('Il file esiste')
else:
    print('Il file non esiste')

# Controllare il tipo
if p.is_file():
    print('È un file')

if p.is_dir():
    print('È una cartella')

if p.is_symlink():
    print('È un collegamento simbolico')
```

Creare Cartelle

```
from pathlib import Path

# Creare una cartella
p = Path('nuova_cartella')
p.mkdir() # Errore se esiste già

# Creare con genitori
p = Path('a/b/c/d')
p.mkdir(parents=True, exist_ok=True) # Crea tutto il percorso, no errore se esiste
```

Creare File

```
from pathlib import Path

# Creare file vuoto
p = Path('nuovo_file.txt')
```

```

p.touch() # Crea il file, no errore se esiste

# Scrivere contenuto
p.write_text('Ciao mondo!') # Scrive testo (UTF-8)
p.write_bytes(b'Dati binari') # Scrive bytes

# Leggere contenuto
contenuto = p.read_text() # Legge tutto il file come stringa
dati = p.read_bytes()     # Legge tutto il file come bytes

```

Rinominare e Spostare

```

from pathlib import Path

p = Path('vecchio_nome.txt')

# Rinominare
p_nuovo = p.rename('nuovo_nome.txt')
print(p_nuovo) # Path('nuovo_nome.txt')

# Spostare in un'altra cartella (rinomina funziona anche per spostamento)
p_nuovo = p.rename(Path('altra_cartella') / 'file.txt')

```

Eliminare File e Cartelle

```

from pathlib import Path

# Eliminare file
p = Path('file.txt')
p.unlink() # Errore se non esiste

# Eliminare con verifica
p.unlink(missing_ok=True) # No errore se non esiste

# Eliminare cartella vuota
d = Path('cartella_vuota')
d.rmdir() # Errore se non è vuota

# Eliminare cartella con contenuto (serve shutil)
import shutil
d = Path('cartella_con_file')
shutil.rmtree(d)

```

Ottenere Info su File

```
from pathlib import Path

p = Path('file.txt')

# Informazioni del file
stat = p.stat()
print(stat.st_size)           # Dimensione in bytes
print(stat.st_mtime)          # Tempo ultima modifica (timestamp)
print(stat.st_atime)          # Tempo ultimo accesso
print(stat.st_ctime)          # Tempo creazione

# Accorciato
print(p.stat().st_size)       # Dimensione direttamente

# Dimensione leggibile
import os
dimensione = p.stat().st_size
print(f"{dimensione / 1024:.2f} KB")
```

Elencare File e Cartelle

```
from pathlib import Path

d = Path('cartella')

# Elencare file della cartella (non ricorsivo)
for item in d.iterdir():
    print(item)

# Solo file
for file in d.glob('*'):
    if file.is_file():
        print(file)

# Solo cartelle
for folder in d.glob('*'):
    if folder.is_dir():
        print(folder)

# Pattern matching
for txt_file in d.glob('*.txt'):
    print(txt_file)
```

```
# Ricorsivo (**/)  
for txt_file in d.rglob('*.txt'):  
    print(txt_file) # Trova .txt in tutte le sottocartelle
```

Leggere Riga per Riga

```
from pathlib import Path  
  
p = Path('file.txt')  
  
# Leggere tutto  
contenuto = p.read_text()  
  
# Leggere riga per riga  
for linea in p.read_text().splitlines():  
    print(linea)  
  
# Con context manager (più efficiente per file grandi)  
with p.open() as f:  
    for linea in f:  
        print(linea.strip())
```

Esempi Pratici

Esempio 1: Manipolazione Base di Percorsi

```
from pathlib import Path  
  
# Creare un percorso  
p = Path('Documenti') / 'Progetti' / 'python' / 'script.py'  
  
# Accedere ai componenti  
print(f"Nome file: {p.name}")           # script.py  
print(f"Cartella: {p.parent}")         # Documenti/Progetti/python  
print(f"Estensione: {p.suffix}")       # .py  
print(f"Percorso assoluto: {p.resolve()}")  
  
# Costruire percorsi derivati  
config_file = p.parent / 'config.json'  
print(config_file) # Documenti/Progetti/python/config.json
```

Esempio 2: Verificare e Creare Struttura di Cartelle

```
from pathlib import Path

def assicura_struttura():
    """Assicura che la struttura di cartelle esista."""
    cartelle = [
        Path('output'),
        Path('output/logs'),
        Path('output/data'),
        Path('output/reports')
    ]

    for cartella in cartelle:
        if not cartella.exists():
            cartella.mkdir(parents=True, exist_ok=True)
            print(f"✓ Creata: {cartella}")
        else:
            print(f"✓ Esiste già: {cartella}")

assicura_struttura()
```

Esempio 3: Elencare File di un Certo Tipo

```
from pathlib import Path

def lista_file_tipo(cartella, estensione):
    """Elenca tutti i file di un tipo in una cartella."""
    p = Path(cartella)

    print(f"\nFile .{estensione} in {p}:\n")

    file_trovati = []
    for file in p.rglob(f'*.{estensione}'):
        print(f"  {file.relative_to(p)}")
        file_trovati.append(file)

    print(f"\nTotale: {len(file_trovati)} file")
    return file_trovati

# Utilizzo (se hai file .py nella cartella corrente)
# lista_file_tipo('.', 'py')
```

Esempio 4: Organizzare File per Estensione

```
from pathlib import Path
import shutil

def organizza_file_per_estensione(cartella_origine, cartella_destinazione):
    """
    Organizza file in sottocartelle basate sull'estensione.
    """
    origen = Path(cartella_origine)
    destino = Path(cartella_destinazione)

    # Creare cartella di destinazione
    destino.mkdir(exist_ok=True, parents=True)

    # Elencare tutti i file
    for file in origen.rglob('*'):
        if file.is_file():
            # Ottenere estensione (senza il punto)
            estensione = file.suffix.lstrip('.') or 'no_estensione'

            # Creare sottocartella per estensione
            subdir = destino / estensione
            subdir.mkdir(exist_ok=True)

            # Spostare il file
            nuovo_percorso = subdir / file.name
            shutil.move(str(file), str(nuovo_percorso))
            print(f"Spostato: {file.name} → {estensione}/")

# Utilizzo:
# organizza_file_per_estensione('./scaricati', './organizzato')
```

Esempio 5: Analizzare Albero di Cartelle

```
from pathlib import Path

def analizza_albero(cartella, indent=0):
    """
    Visualizza ricorsivamente la struttura di cartelle.
    """
    p = Path(cartella)

    # Non analizzare se non è una cartella
```

```

if not p.is_dir():
    return

# Elencare contenuto
try:
    items = sorted(p.iterdir())
except PermissionError:
    print(" " * indent + "❌ Accesso negato")
    return

for item in items:
    if item.is_dir():
        print(" " * indent + f"📁 {item.name}/")
        analizza_albero(item, indent + 1)
    else:
        dimensione = item.stat().st_size
        print(" " * indent + f"📄 {item.name} ({dimensione} bytes)")

# Utilizzo:
# analizza_albero('.')

```

Esempio 6: Cercare File con Criteri

```

from pathlib import Path

def cerca_file(cartella, **criteri):
    """
    Cerca file con criteri specifici.

    Criteri supportati:
    - estensione: '.txt', '.py'
    - dimensione_min: 1024 (bytes)
    - dimensione_max: 1048576 (bytes)
    """
    p = Path(cartella)
    risultati = []

    for file in p.rglob('*'):
        if not file.is_file():
            continue

        # Controllare estensione
        if 'estensione' in criteri:
            if file.suffix != criteri['estensione']:
                continue

```

```

# Controllare dimensione minima
if 'dimensione_min' in criteri:
    if file.stat().st_size < criteri['dimensione_min']:
        continue

# Controllare dimensione massima
if 'dimensione_max' in criteri:
    if file.stat().st_size > criteri['dimensione_max']:
        continue

risultati.append(file)

return risultati

# Utilizzo:
# file_grandi = cerca_file('.', estensione='.txt', dimensione_min=1000)
# for f in file_grandi:
#     print(f)

```

Esempio 7: Copiare Struttura di Cartelle

```

from pathlib import Path
import shutil

def copia_struttura(origine, destinazione):
    """
    Copia l'intero albero di cartelle (solo struttura, non file).
    """
    o = Path(origine)
    d = Path(destinazione)

    # Creare cartella di destinazione
    d.mkdir(parents=True, exist_ok=True)

    # Copiare ricorsivamente
    for cartella in o.rglob('*/*'):
        rel_path = cartella.relative_to(o)
        nuova_cartella = d / rel_path
        nuova_cartella.mkdir(exist_ok=True)
        print(f"✓ Creata: {nuova_cartella}")

```

```
# Utilizzo:
# copia_struttura('./sorgente', './copia')
```

Esempio 8: Statistiche di Cartella

```
from pathlib import Path

def statistiche_cartella(cartella):
    """
    Mostra statistiche su una cartella.
    """
    p = Path(cartella)

    num_file = 0
    num_cartelle = 0
    dimensione_totale = 0
    estensioni = {}

    for item in p.rglob('*'):
        if item.is_file():
            num_file += 1
            dimensione_totale += item.stat().st_size

            # Contare estensioni
            ext = item.suffix or 'no_estensione'
            estensioni[ext] = estensioni.get(ext, 0) + 1

        elif item.is_dir():
            num_cartelle += 1

    print(f"Statistiche di {p}:")
    print(f"  File: {num_file}")
    print(f"  Cartelle: {num_cartelle}")
    print(f"  Dimensione totale: {dimensione_totale / 1024 / 1024:.2f} MB")
    print(f"  Estensioni:")
    for ext, count in sorted(estensioni.items(), key=lambda x: x[1], reverse=True):
        print(f"    {ext}: {count}")

# Utilizzo:
# statistiche_cartella('.')
```

Casi d'Uso Reali

Caso 1: Backup Automatico

```
from pathlib import Path
from datetime import datetime
import shutil

def crea_backup(cartella_origine):
    """
    Crea un backup di una cartella con timestamp.
    """
    origine = Path(cartella_origine)

    if not origine.exists():
        print(f"Errore: {origine} non esiste")
        return

    # Creare nome cartella backup
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    backup_dir = Path('backup') / f"{origine.name}_{timestamp}"

    # Creare backup
    backup_dir.mkdir(parents=True, exist_ok=True)

    # Copiare file
    for file in origine.rglob('*'):
        if file.is_file():
            rel_path = file.relative_to(origine)
            dest_file = backup_dir / rel_path
            dest_file.parent.mkdir(parents=True, exist_ok=True)
            shutil.copy2(file, dest_file)

    print(f"✓ Backup creato: {backup_dir}")
    return backup_dir

# Utilizzo:
# crea_backup('./miei_file')
```

Caso 2: Pulizia di File Temporanei

```
from pathlib import Path
from datetime import datetime, timedelta
```

```

def pulisci_file_vecchi(cartella, giorni=7):
    """
    Elimina file non modificati da N giorni.
    """
    p = Path(cartella)
    limite_tempo = datetime.now() - timedelta(days=giorni)

    file_eliminati = 0

    for file in p.rglob('*'):
        if not file.is_file():
            continue

        mtime = datetime.fromtimestamp(file.stat().st_mtime)

        if mtime < limite_tempo:
            file.unlink()
            print(f"Eliminato: {file.name}")
            file_eliminati += 1

    print(f"Totale eliminati: {file_eliminati}")

# Utilizzo:
# pulisci_file_vecchi('./temp', giorni=7)

```

Caso 3: Rinomina Batch di File

```

from pathlib import Path

def rinomina_batch(cartella, pattern_old, pattern_new):
    """
    Rinomina file in batch (semplice find-replace).
    """
    p = Path(cartella)

    for file in p.glob('*'):
        if file.is_file():
            nuovo_nome = file.name.replace(pattern_old, pattern_new)

            if nuovo_nome != file.name:
                file.rename(file.parent / nuovo_nome)
                print(f"{file.name} → {nuovo_nome}")

```

```
# Utilizzo:
# rinomina_batch('.', 'old_', 'new_')
```

Caso 4: Validare Struttura di Progetto

```
from pathlib import Path

def valida_struttura_progetto():
    """
    Valida che la struttura di un progetto sia corretta.
    """
    cartelle_richieste = [
        Path('src'),
        Path('tests'),
        Path('docs'),
        Path('docs/images')
    ]

    file_richiesti = [
        Path('README.md'),
        Path('setup.py'),
        Path('requirements.txt')
    ]

    print("Validazione struttura progetto:")

    # Controllare cartelle
    for cartella in cartelle_richieste:
        if cartella.exists() and cartella.is_dir():
            print(f" ✓ Cartella {cartella} esiste")
        else:
            print(f" X Cartella {cartella} MANCA")
            cartella.mkdir(parents=True, exist_ok=True)

    # Controllare file
    for file in file_richiesti:
        if file.exists():
            print(f" ✓ File {file} esiste")
        else:
            print(f" X File {file} MANCA")

# Utilizzo:
# valida_struttura_progetto()
```

Esercizi

Esercizio 1: Contare File per Estensione

Crea una funzione che conta quanti file di ogni estensione ci sono in una cartella.

```
def conta_file_per_estensione(cartella):  
    """  
    Conta file per estensione.  
  
    TODO: Implementare  
    """  
    pass  
  
# Test  
# risultato = conta_file_per_estensione('.')  
# print(risultato)  
# Output esempio: {'.py': 5, '.txt': 3, '.md': 2}
```

Soluzione:

```
from pathlib import Path  
  
def conta_file_per_estensione(cartella):  
    p = Path(cartella)  
    contatori = {}  
  
    for file in p.rglob('*'):  
        if file.is_file():  
            ext = file.suffix or 'no_estensione'  
            contatori[ext] = contatori.get(ext, 0) + 1  
  
    return contatori
```

Esercizio 2: File più Recente

Crea una funzione che trova il file modificato più recentemente in una cartella.

```
from pathlib import Path  
from datetime import datetime  
  
def file_piu_recente(cartella):  
    """  
    Trova il file modificato più recentemente.  
    """
```

```

    TODO: Implementare
    """

    pass

# Test
# file = file_piu_recente('.')
# print(f"File più recente: {file}")

```

Soluzione:

```

from pathlib import Path
from datetime import datetime

def file_piu_recente(cartella):
    p = Path(cartella)
    file_lista = [(f, f.stat().st_mtime) for f in p.rglob('*') if f.is_file()]

    if not file_lista:
        return None

    return max(file_lista, key=lambda x: x[1])[0]

```

Esercizio 3: Trovare File Duplicati (per Nome)

Crea una funzione che trova file con lo stesso nome in sottocartelle diverse.

```

def trova_duplicati(cartella):
    """
    Trova file con lo stesso nome in cartelle diverse.

    TODO: Implementare
    """

    pass

# Test
# duplicati = trova_duplicati('.')
# for nome, percorsi in duplicati.items():
#     print(f"{nome}: {percorsi}")

```

Soluzione:

```

from pathlib import Path

```

```
def trova_duplicati(cartella):
    p = Path(cartella)
    nomi = {}

    for file in p.rglob('*'):
        if file.is_file():
            nome = file.name
            if nome not in nomi:
                nomi[nome] = []
            nomi[nome].append(file)

    # Tornare solo i nomi con duplicati
    return {nome: file_list for nome, file_list in nomi.items() if len(file_list) > 1}
```

Esercizio 4: Calcolare Dimensione Cartella

Crea una funzione che calcola la dimensione totale di una cartella.

```
def dimensione_cartella(cartella):
    """
    Calcola la dimensione totale di una cartella in bytes.

    TODO: Implementare
    """
    pass

# Test
# size = dimensione_cartella('.')
# print(f"Dimensione: {size / 1024 / 1024:.2f} MB")
```

Soluzione:

```
from pathlib import Path

def dimensione_cartella(cartella):
    p = Path(cartella)
    dimensione_totale = 0

    for file in p.rglob('*'):
        if file.is_file():
            dimensione_totale += file.stat().st_size

    return dimensione_totale
```

Esercizio 5: Cercare File per Contenuto

Crea una funzione che cerca file di testo contenenti una stringa specifica.

```
def cerca_in_file(cartella, testo):  
    """  
    Trova file di testo contenenti un testo specifico.  
  
    TODO: Implementare  
    """  
    pass  
  
# Test  
# risultati = cerca_in_file('.', 'def ')  
# for file, linee in risultati.items():  
#     print(f"{file}: {linee} linee")
```

Soluzione:

```
from pathlib import Path  
  
def cerca_in_file(cartella, testo):  
    p = Path(cartella)  
    risultati = {}  
  
    for file in p.rglob('*.txt'): # Limitare a file di testo  
        try:  
            contenuto = file.read_text(encoding='utf-8', errors='ignore')  
            if testo in contenuto:  
                linee_trovate = [i+1 for i, linea in enumerate(contenuto.split('\n'))  
                                if testo in linea]  
                risultati[file] = linee_trovate  
        except:  
            pass  
  
    return risultati
```

pathlib vs os.path

Confronto Diretto

Operazione	os.path	pathlib
Creare percorso	<code>os.path.join('a', 'b')</code>	<code>Path('a') / 'b'</code>
Estensione	<code>os.path.splitext('f.txt')[1]</code>	<code>Path('f.txt').suffix</code>
Nome file	<code>os.path.basename('/a/b/c.txt')</code>	<code>Path('/a/b/c.txt').name</code>
Cartella padre	<code>os.path.dirname('/a/b/c.txt')</code>	<code>Path('/a/b/c.txt').parent</code>
Esiste?	<code>os.path.exists(path)</code>	<code>path.exists()</code>
È file?	<code>os.path.isfile(path)</code>	<code>path.is_file()</code>
È cartella?	<code>os.path.isdir(path)</code>	<code>path.is_dir()</code>
Creare cartella	<code>os.makedirs(path)</code>	<code>path.mkdir()</code>
Elencare file	<code>os.listdir(path)</code>	<code>path.iterdir()</code>
Leggere file	<code>open(path).read()</code>	<code>path.read_text()</code>

Esempi di Migrazione

```
# VECCHIO (os.path)
import os

file_path = os.path.join('data', 'input.txt')
if os.path.exists(file_path):
    size = os.path.getsize(file_path)
    content = open(file_path).read()

# NUOVO (pathLib)
from pathlib import Path

file_path = Path('data') / 'input.txt'
if file_path.exists():
    size = file_path.stat().st_size
    content = file_path.read_text()
```

Quando Usare Cosa?

✅ Usare pathlib:

- Codice nuovo
- Manipolazione di percorsi
- Operazioni su file singoli
- Codice moderno e leggibile

✅ Usare os.path:

- Codice legacy da mantenere
- Casi specifici e particolari
- Quando serve compatibilità estrema

Tips e Tricks

Creare Percorso Temporaneo

```
from pathlib import Path
import tempfile

# Cartella temporanea
with tempfile.TemporaryDirectory() as tmpdir:
    p = Path(tmpdir) / 'file.txt'
    p.write_text('Dati temporanei')
```

Percorsi Relativi vs Assoluti

```
from pathlib import Path

p = Path('cartella/file.txt')

# Relativo rispetto alla cartella attuale
print(p.relative_to(Path.cwd()))

# Relativo rispetto a un'altra cartella
print(p.relative_to(Path('cartella'))) # 'file.txt'
```

Glob Patterns

```
from pathlib import Path

p = Path('.')

# Tutti i file
p.glob('*')

# Python files only
p.glob('*.py')

# Ricorsivo
p.glob('**/*.py')

# Pattern complessi
p.glob('**/test_*.py')
```

Context Manager con Path

```
from pathlib import Path

p = Path('file.txt')

# Open direttamente
with p.open() as f:
    for linea in f:
        print(linea)

# O read_text
contenuto = p.read_text()
```

Avvertenze

Windows vs Unix

```
from pathlib import Path, PureWindowsPath, PurePosixPath

# Questo funziona sempre
p = Path('a/b/c.txt')
```

```
# Questo è specifico dell'OS
pw = PureWindowsPath('a\\b\\c.txt')
pp = PurePosixPath('a/b/c.txt')
```

Percorsi Simbolici

```
from pathlib import Path

p = Path('collegamento')

if p.is_symlink():
    p_reale = p.resolve() # Segue il collegamento
```

Permessi

Alcune operazioni possono fallire se non hai permessi.

```
from pathlib import Path

p = Path('file_protetto.txt')

try:
    p.unlink()
except PermissionError:
    print("Non hai permessi di eliminazione")
```

File in Uso

Non puoi eliminare file in uso su alcuni OS (Windows).

```
from pathlib import Path

p = Path('file_in_uso.txt')

try:
    p.unlink()
except OSError as e:
    print(f"File in uso: {e}")
```

Conclusione

`pathlib` è il modo **moderno e consigliato** per lavorare con percorsi in Python. Offre:

- ✓ Sintassi intuitiva con `/`
- ✓ Metodi orientati agli oggetti
- ✓ Cross-platform automatico
- ✓ Codice più leggibile
- ✓ Built-in senza installazione

Migra i tuoi progetti a pathlib! 🚀

Risorse Utili

- **Documentazione ufficiale:** <https://docs.python.org/3/library/pathlib.html>
- **PEP 428:** <https://www.python.org/dev/peps/pep-0428/> (proposta originale)
- **Guida RealPython:** <https://realpython.com/python-pathlib/>
- **Built-in Path methods:** <https://docs.python.org/3/library/pathlib.html#pathlib.Path>